



DATA STRUCTURES

CONTENTS



23. Balanced Binary Search Trees and AVL Trees

T.V. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept of an important ADT – the Tree ADT and also the special type of tree the binary search tree. In this module we will discuss one very important type of binary search tree that is the balanced binary search tree and one example of balanced binary search tree namely the AVL trees.

Learning Objectives

The learning objectives of the module are as follows:

- To understand the concept of Balanced Binary Search Trees
- To discuss the properties of AVL Trees
- To describe Single Rotation of AVL Trees
- To discuss the Double Rotation of AVL Trees

23.1 Introduction to Balanced Binary Search Trees

As a tree becomes more unbalanced, running time of the search operation increases from $O(\log n)$ to $O(n)$ because as the tree gets more unbalanced the shape starts becoming a list. Binary Search Trees are an important data structure used in many applications needing the search operation are fast if they're shallow or in other words the tree is a complete tree. The disadvantage of an unbalanced binary search tree is that its height can be as large as $N-1$. This essentially means that when one branch is much longer than the other, the time needed to perform insertion, deletion and search operations is much more. Now we need to define the concept of a "sort of" complete tree where by "sort of" we mean a binary search tree that is as balanced as possible. In other words we want to keep the binary search tree balanced as nodes are added/removed, so searching/insertion remain efficient.

A class of binary search trees is said to be **balanced**, if each of the three dictionary operations searching, inserting and deleting of keys for a tree with n keys can always (in the worst case) be carried out in $O(\log n)$ steps.

23.1.1 Approaches to Balancing Trees

There are many approaches that can be adopted for balancing trees. The first and simplest method is we do not do balancing. However in this case we may end up with some nodes being very deep. The other extreme of the simple policy is to maintain a strict balance that is the tree must always be balanced perfectly. The next approach is what is called pretty good balance where we allow a little out of balance. The fourth and final approach is adjusting the balance during access – the so called self-adjusting trees.

23.2 Balanced Binary Search Trees

There are two basic kinds of balanced binary search trees. **Height balanced trees** where we ensure that the height of siblings are "approximately the same". The other type is the **Weight-balanced trees** where we ensure that the number of descendants of sibling nodes are "approximately the same". "Tree rotations" are used to maintain balance on insert/delete

Now let us define the concept of balance – A *binary tree* is **balanced**(here unless otherwise specified balanced implies height balanced) if the difference in height between any node's left and right subtree is ≤ 1 .

$$\text{balance} = \text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$$

By convention the height of a “null” subtree is assumed to be -1. If the balance is zero everywhere then the tree is said to be perfectly balanced. If the balance is small everywhere that is at each and every node then the tree is balanced enough to ensure operations will take time of the $O(\log n)$. This is because the maximum depth of the tree can be only $1.44 \log n$.

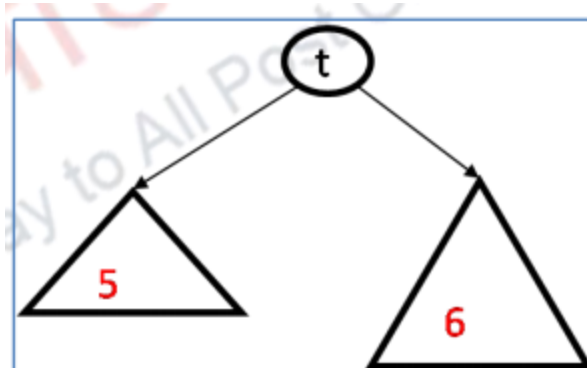


Figure 23.1 Balanced Tree

Perfect Balance is said to occur if we want a complete tree after every operation. This means that the tree is full except possibly in the lower right. However maintaining perfect balance is expensive. For example (Figure 23.2), insert 2 in the tree on the left and then rebuild as a complete tree

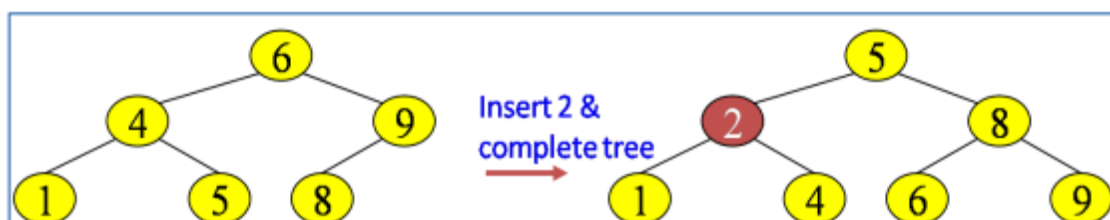


Figure 23.2 Perfect Balancing

23.2.1 Prevent the degeneration of the Binary Search Tree (BST) :

A BST can be set up to maintain balance during every update operation typically insertions and removals. Our objective is to keep the height of a binary search tree $O(\log N)$. This allows us to achieve a worst-case runtime of $O(\log n)$ for searching, inserting and deleting.

Types of BST which maintain the optimal performance are the following:

- **AVL trees**
- **Red-Black trees**
- **Splay trees**

AVL Trees are BSTs that maintain **height balance**. For each node, the difference in height of its two subtrees is in the range -1 to 1

Red-black tree is a binary version of a 2-3-4 tree. Here the nodes have a 'color' attribute: **BLACK** or **RED**. The tree maintains a balance measure called the **BLACK height**

A **splay tree** are self-adjusting type of BST with the additional property that **recently accessed elements are quick to be access again**. The basic operations such as insertion, look-up and removal takes place in $O(\log n)$ amortized time. Insert/find always rotates node to the root.

23.3 AVL Tree

We will first discuss an AVL tree which is essentially a binary search tree with a *balance* condition. AVL is named after its inventors: **Adel'son-Vel'skii** and **Landis**. The AVL tree **approximates the ideal tree** (completely balanced tree) since the AVL Tree maintains a height close to the minimum (Figure 23.3) However It is not a requirement that all leaves of an AVL tree be on the same or adjacent level.



Figure 23.3 Valid AVL Tree

23.3.1 AVL – Good but not Perfect Balance

As we have already discussed AVL tree is a binary search tree that has an additional **height constraint**: For each node x in the tree, $\text{Height}(x.\text{left})$ differs from $\text{Height}(x.\text{right})$ by at most 1. If this height constraint is satisfied then the height of the tree is $O(\log n)$.

As we already discussed the Balance factor of a node which is defined below:

$\text{height}(\text{left sub-tree}) - \text{height}(\text{right sub-tree})$

The AVL tree has balance factor calculated at every node, where for every node, the heights of the left and right sub-trees can differ by no more than 1. Each node stores the current heights at each node. The height of an AVL tree storing n keys is $O(\log n)$.

The height of a node is now specified. The height of a leaf is 0. The height of a null tree is assumed to be -1. In general, the height of an internal node is the maximum height of its children plus 1 (Figure 23.4). The height of any node is the maximum of the heights of its two sub-trees. $\text{Maximum of } (H-1, H-2) + 1 = H-1+1 = H$

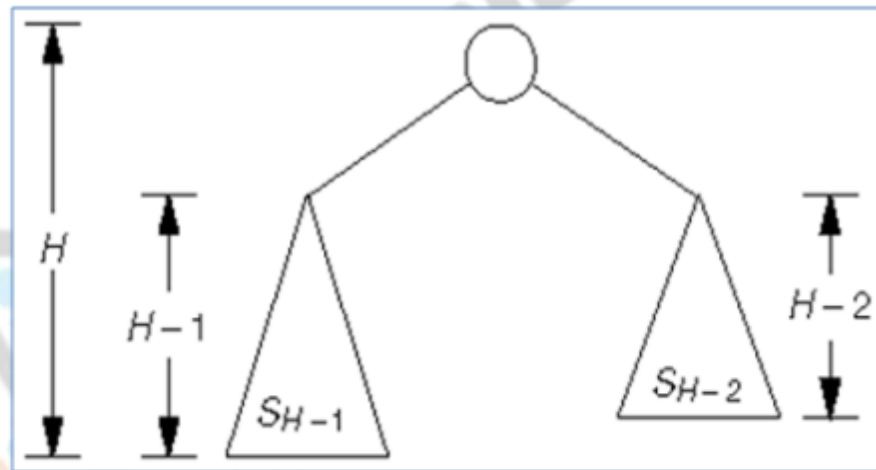


Figure 23.4 Height of the Node

Now let us look at an example of an AVL Tree (Figure 23.5).

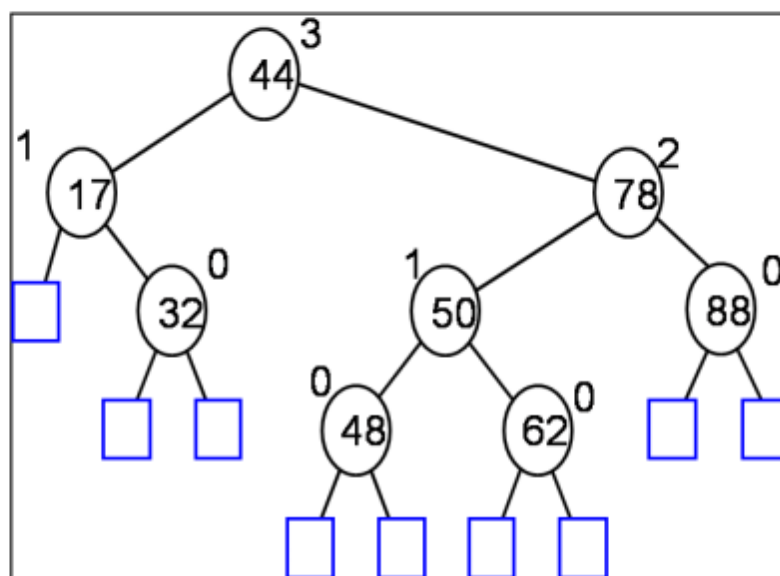


Figure 23.5 Example of an AVL Tree with Heights shown

An example of an AVL tree where the heights are shown next to the nodes is given in Figure 23.5. Now remember an AVL tree is a binary search tree. The height of the left and right sub-trees of the root differ by at most 1. Now each left and right sub-trees are themselves AVL trees. An AVL Tree T is a binary search tree such that for every internal node v of T , the heights of the children of v can differ by at most 1.

To sum up :

Height of empty node = -1, Height of leaf = 0 and Height of a node = Max (Height of left subtree, Height of right subtree) +1

23.3.2 Balance Factor of AVL Trees

For each AVL tree node, the difference between the heights of its left and right sub-trees is either -1, 0 or +1 is called the balance factor of a node. The $\text{balanceFactor} = \text{height}(\text{left sub-tree}) - \text{height}(\text{right sub-tree})$. If $\text{balanceFactor} > 1$ or < -1 then the tree is too unbalanced, and needs 'rearranging' to make it more balanced. When we have a positive balance factor the node is said to be "heavy on the left" that is the height of the left sub-tree is greater than the height of the right sub-tree. When we have negative balance factor, the node is "heavy on the right". Figure 23.6 shows an example where the heights and balance factors are shown next to the nodes

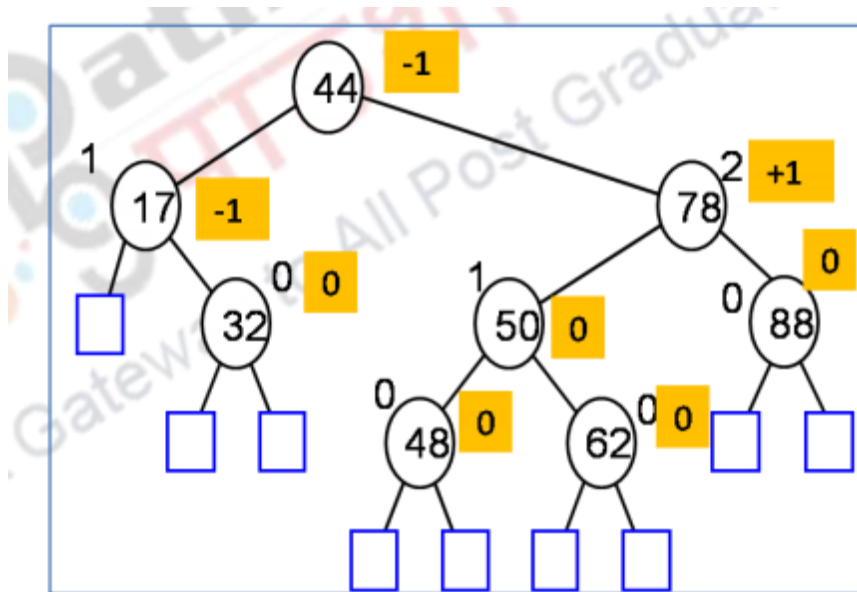


Figure 23.6 AVL Tree with heights and balance factors shown

Now the balance factor of an AVL tree can be either 0, +1 or -1. Due to insertions or deletions the balance factor can change from 0 to +1 or -1. In this case there is no imbalance. When the balance factor changes from +1/-1 to +2/-2 then comes the need to rebalance so that the balance factor is back to 0, +1 or -1.

23.4 Properties of AVL Trees

The depth of a typical node in an AVL tree approaches the optimal value possible which is $\log N$. Consequently, all searching operations in an AVL tree have

logarithmic worst-case bounds. An update (insert or remove) in an AVL tree could destroy the balance. Therefore we go about rebalancing before the operation can be considered complete. This time taken for rebalancing during update operation is the price we pay for making the search operation faster. It is to be noted that after an insertion, only nodes that are on the path from the insertion point to the root can have their balances altered. This affects the way we carry out rebalancing.

23.5 Rotations

Insertion or deletion operation involves adding or deleting only a single node at a time. This essentially means that the height of some sub-tree can change by at most 1. If the AVL tree property is violated at a node x , it means that the heights of $\text{left}(x)$ and $\text{right}(x)$ differ by exactly 2. Now rotations will have to be applied to x to restore the AVL tree property. The rotation is generally a $O(1)$ operation.

In order to restore the balance of an unbalanced tree, three adjacent nodes are involved in the rotation. The deepest unbalanced node, is the node that requires rotation to rebalance the tree. This node is either the ancestor of a deleted node (in case of deletion) or of the inserted node (in case of insertion) and is the node whose balance factor has changed to -2 or $+2$. This is shown using an example of insertion in Figure 23.7. Here the element 35 is inserted. Therefore the three nodes that take part in the rotation are the deepest unbalanced node (here 45), the ancestor of the inserted node (40) and the inserted node (35).

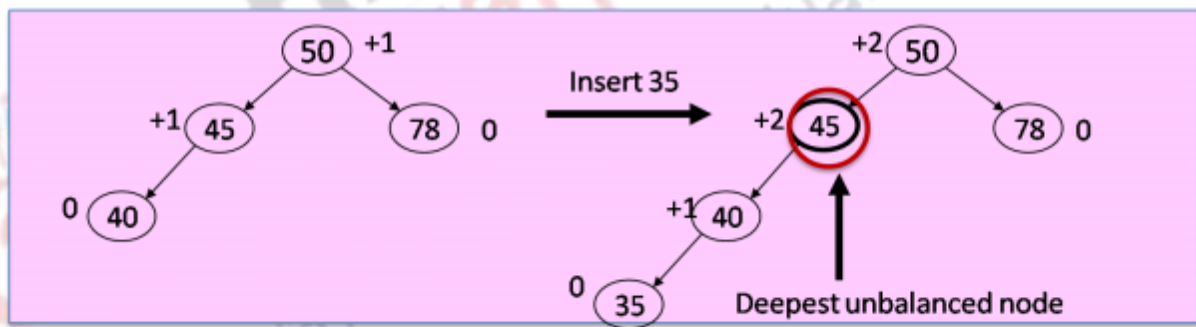


Figure 23.7 Restoring balance of the tree

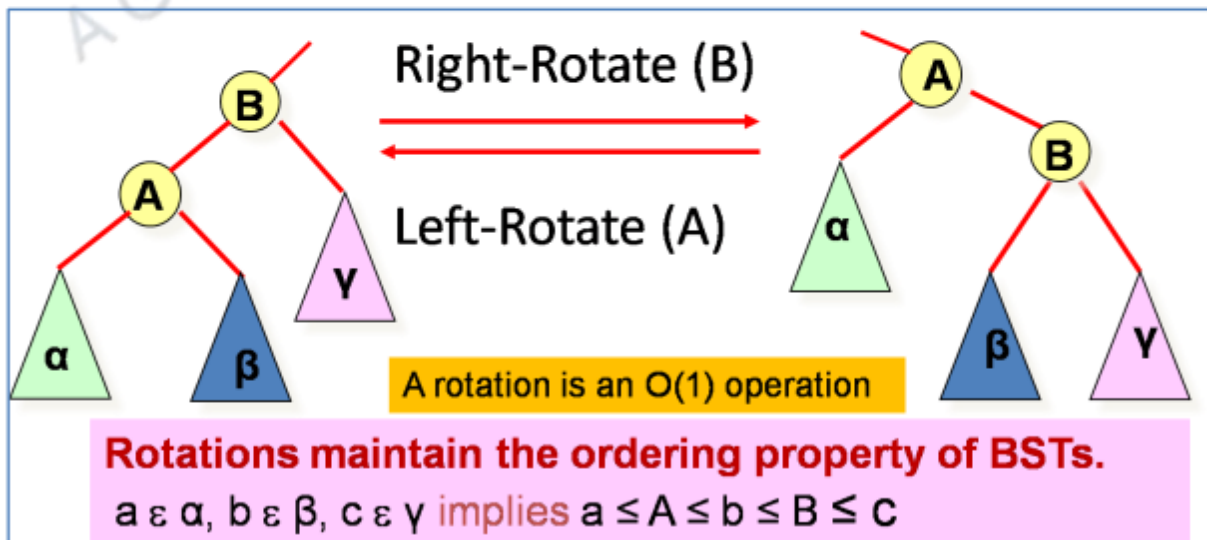


Figure 23.8 Maintaining BST Ordering Property

23.5.1 BST Ordering Property after a Rotation

A rotation does not affect the ordering property of a BST (binary search tree). This is explained in Figure 23.8. Let us assume $a \in \alpha$, $b \in \beta$ and $c \in \gamma$. This implies that $a \leq A \leq b \leq B \leq c$ according to the BST tree property. It can be seen that even after rotation the same property is maintained. We will explain the actual procedure of rotation in the next section.

23.5.2 Concept of Rotation and BST Property

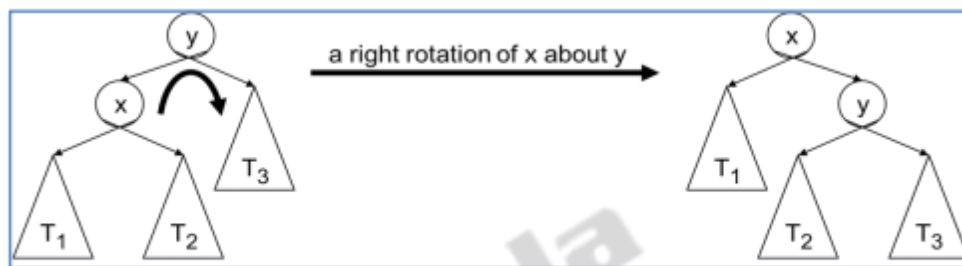


Figure 23.9 Rotation and BST Property

Now let us discuss rotation in terms of subtrees. BST ordering property requirement means that on the left hand side tree $T_1 < x < y, x < T_2 < y$ and $x < y < T_3$. Now we carry out a right rotation of x about y (Figure 23.9). This means that the following changes are made:

- y which was the parent with x as left child ($x < y$) before rotation now becomes the right child of parent x retaining the $x < y$ property.
- T_2 which was the right sub-tree of x ($x < T_2 < y$) before rotation now becomes the left sub-tree of y still maintaining the same relation.
- Therefore after rotation the same property $T_1 < x < y, x < T_2 < y$ and $x < y < T_3$ is maintained.

Though we have seen an example of right rotation, the same is true for left rotation.

23.6 Different Cases of Rotations

There are basically two types of rotations based on whether we carry out single or double rotations. We also classify rotations based on where and on which subtree (left or right) the violation or the imbalance occurs. Here we also use the term outside grandchild that is the left grandchild of a left child (left-left) or the right grandchild of a right child (right-right). We have an inside grandchild when we have left grandchild of a right child (left-right) or the right grandchild of a left child (right-left). Therefore considering these aspects we have the following four cases of rotations:

Violation may occur during an insertion into

Case1. left sub-tree of left child (**Single Right Rotation**)

Case 2. right sub-tree of right child (**Single Left Rotation**)

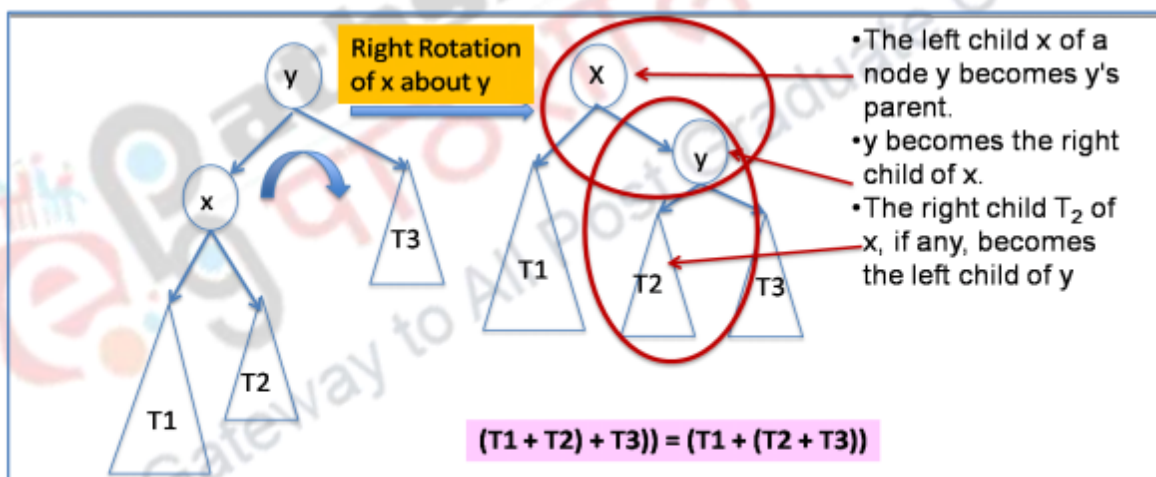
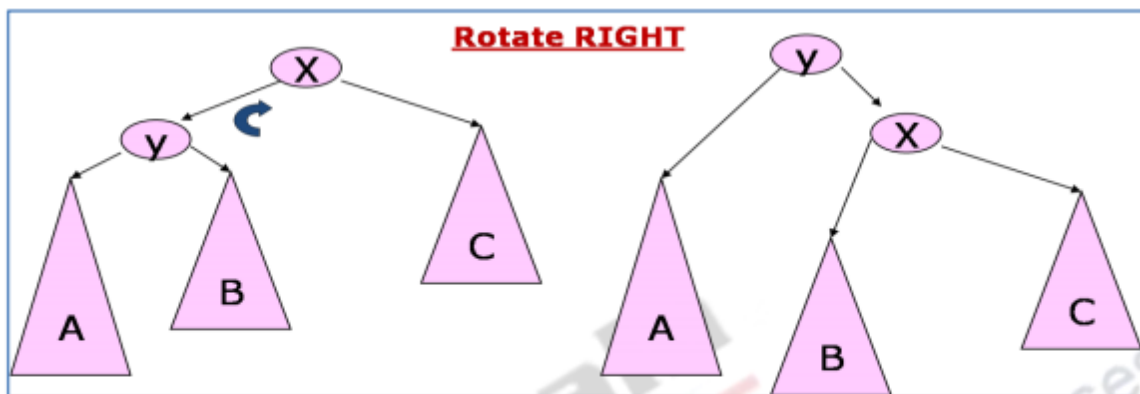
Case 3. right sub-tree of left child (**Double Left – Right Rotation**)

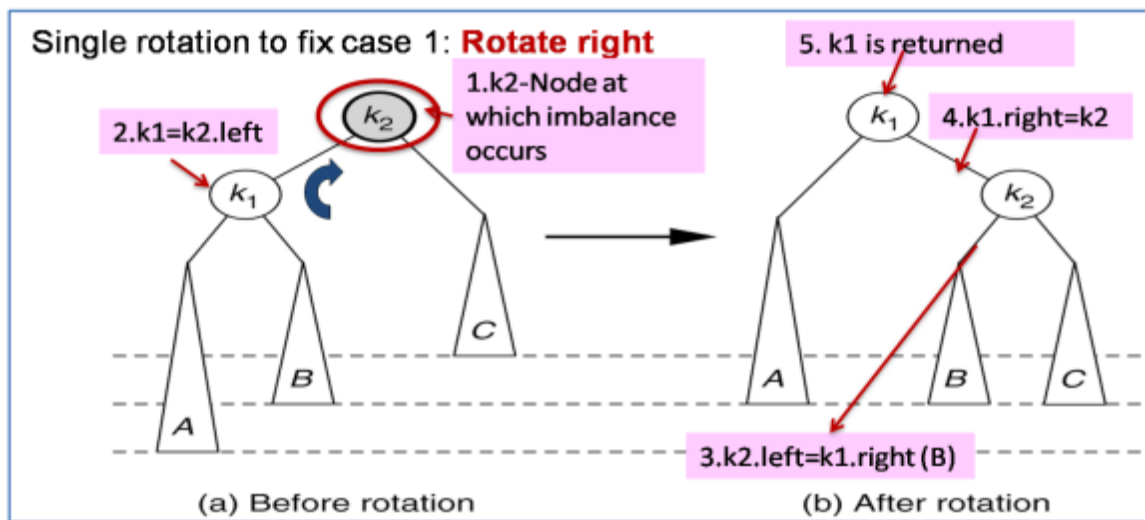
Case 4. left sub-tree of right child (**Double Right – Left Rotation**)

23.6.1 Single Rotation

The simplest type of rotation is single rotation. Case 1 and case 2 are both cases of single rotation. Single rotation switches the roles of the parent and child while maintaining the search order. Here we rotate between a node and its child. While the child becomes parent, parent becomes right child in case 1 and left child in case 2. The result is a binary search tree that satisfies the AVL property. Now let us discuss in detail the above two cases.

23.6.2 Single Right Rotation





(c)

Figure 23.10 Single Right Rotation

Figure 23.10 shows the details of case 1. In this case the left sub-tree of the left child of X violates the property. In Figure 23.10 (a) this means sub-tree A, the left sub-tree of the left child Y of X violates the property. We need to carry out single right rotation of X about Y where X now becomes the right child of Y, and the right sub-tree of Y becomes the left sub-tree of X.

Figure 23.10 (b) shows the three steps namely

- the left child x of a node y becomes y's parent.
- y becomes the right child of x.
- The right subtree T2 of x, if any, becomes the left child of y

Here the notation $((T_1 + T_2) + T_3)$ indicates that the subtrees T_1 and T_2 are children of the same parent and that the combined subtree of $(T_1 + T_2)$ and subtree T_3 are children of the root. Moreover the sub-trees change positions from $((T_1 + T_2) + T_3)$ before the rotation to positions $(T_1 + (T_2 + T_3))$ after the rotation still maintaining the BST property.

Figure 23.10 (c) shows the steps more in detail.

- The node k2 is the node at which the imbalance occurs and the balance factor is violated since the height of subtree c has height difference of 2 with the subtree A.
- The left child of k2 is given by k1=k2.left.
- The new left sub-tree of k2 is now is k1's right sub-tree (B) that is k2.left=k1.right
- The new right sub-tree of k1 is k2 that is k1.right=k2
- Now we return the root of the new balanced tree that is k1.

The algorithm is given below in Figure 23.10 (d) (notations from Figure 23.10 (b))

```

private TreeNode rightRotate(TreeNode oldParent) {

    // 1. detach left child's (x's) right subtree (T2)

    TreeNode orphan= oldParent.left.right;

    // 2. consider left child (x) to be the new parent

    TreeNode newParent = oldParent.left;

    // 3. attach old parent (y) onto right of new parent (x)

    newParent.right = oldParent;

    // 4. attach new parent's (x's) right subtree (T2)as

    //    left subtree of old parent (y)

    oldParent.left = orphan;

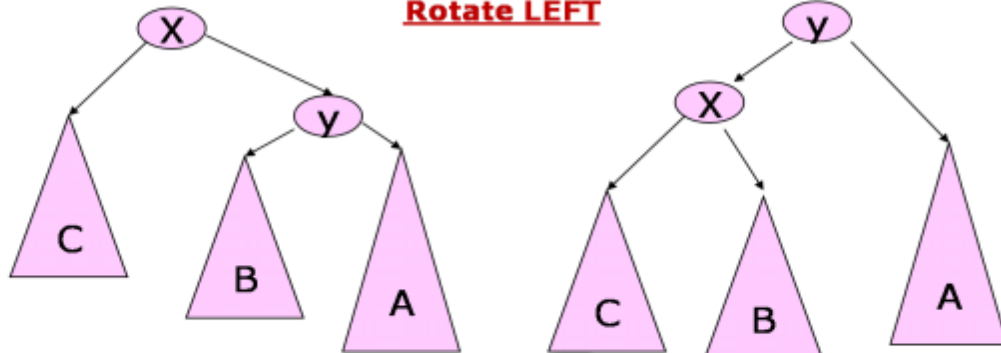
    oldParent.height = height(oldParent); // update nodes'
    newParent.height = height(newParent); // height values
    return newParent;
}

```

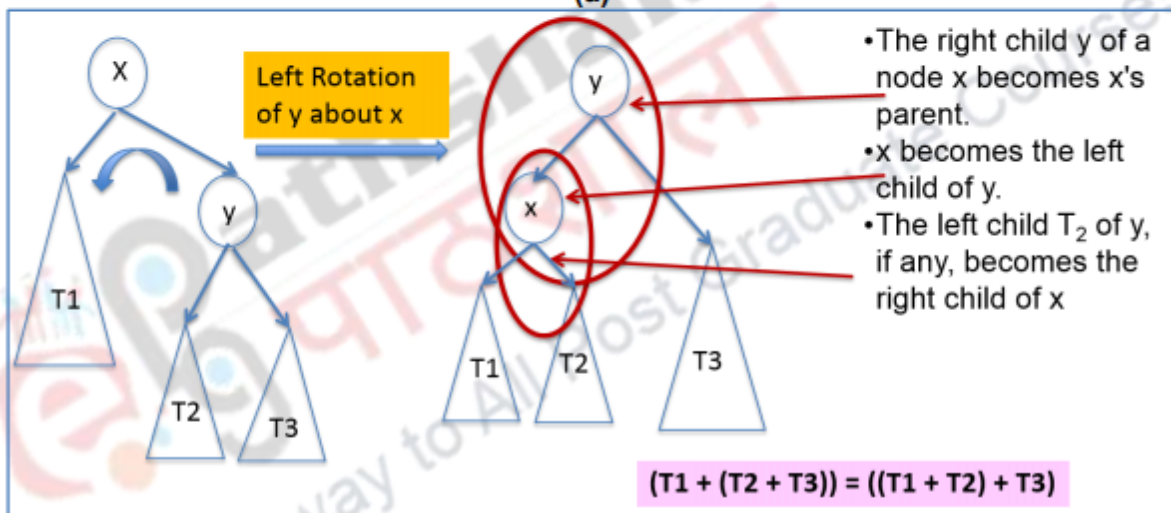
Figure 23.10 (d) Algorithm for Single Right Rotation 23.6.3 Single Left Rotation

23.6.3 Single Left Rotation

Case 2 – The right subtree of the right child of X violates the property. Rotate LEFT

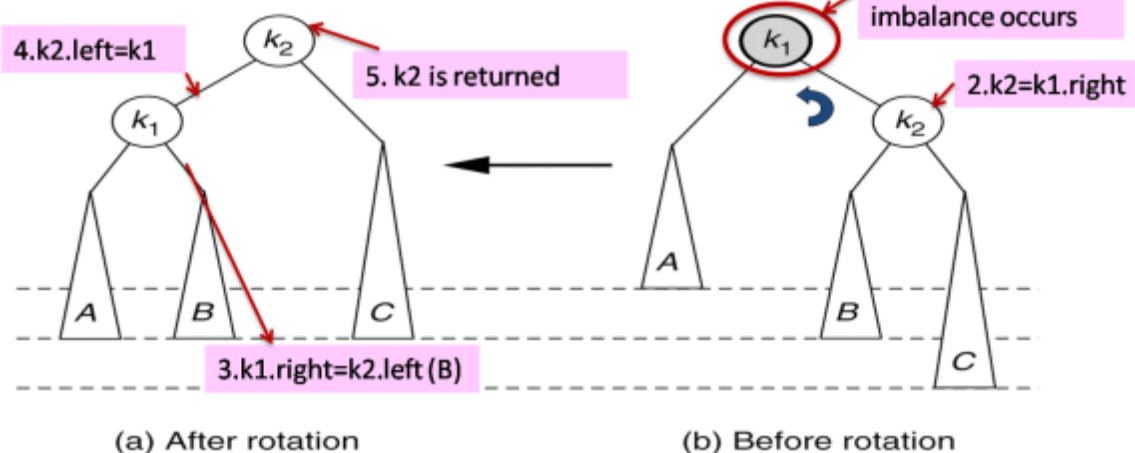


(a)



(b)

Symmetric single rotation to fix case 2 : Rotate left



(c)

Figure 23.11 Single Left Rotation

Figure 23.11 shows the details of case 2. In this case the right sub-tree of the right child of X violates the property. In Figure 23.11 (a) this means sub-tree A, the right sub-tree of the right child Y of X violates the property. We need to carry out single left rotation of Y about X where X now becomes the left child of Y, and the left sub-tree of Y becomes the right sub-tree of X.

Figure 23.11 (b) shows the three steps namely

- The right child y of a node x becomes x's parent.
- x becomes the left child of y.
- The left child T2 of y, if any, becomes the right child of x

Moreover the sub-trees change positions from $(T1 + (T2 + T3))$ before the rotation to positions $((T1 + T2) + T3)$ after the rotation still maintaining the BST property.

Figure 23.10 (c) shows the steps more in detail.

- The node k1 is the node at which the imbalance occurs and the balance factor is violated where the height of subtree c differs with the height of subtree A by more than one.
- The right child of k2 is given by $k2 = k1.\text{right}$.
- The new right sub-tree of k1 is now is k2's left sub-tree (B) that is $k1.\text{right} = k2.\text{left}$ (B)
- The new left sub-tree of k2 is k1 that is $k2.\text{left} = k1$
- Now we return the root of the new balanced tree that is k2.
- The algorithm is given below in Figure 23.11 (d) (notations from Figure 23.11 (b))

```
private TreeNode leftRotate(TreeNode oldParent) {  
  
    // 1. detach right child's (y) left subtree (T2)  
  
    TreeNode orphan = oldParent.right.left;  
  
    // 2. consider right child (y) to be the new parent  
  
    TreeNode newParent = oldParent.right;  
  
    // 3. attach old parent (x) onto left of new parent (y)  
  
    newParent.left = oldParent;  
  
    // 4. attach new parent's old left subtree (T2) //asright subtree of old parent (x)  
  
    oldParent.right = orphan;  
  
    oldParent.height = height(oldParent); // update nodes' newParent.height =  
height(newParent); // height values return newParent;  
  
}
```

Figure 23.11 (d) Algorithm for Single Left Rotation

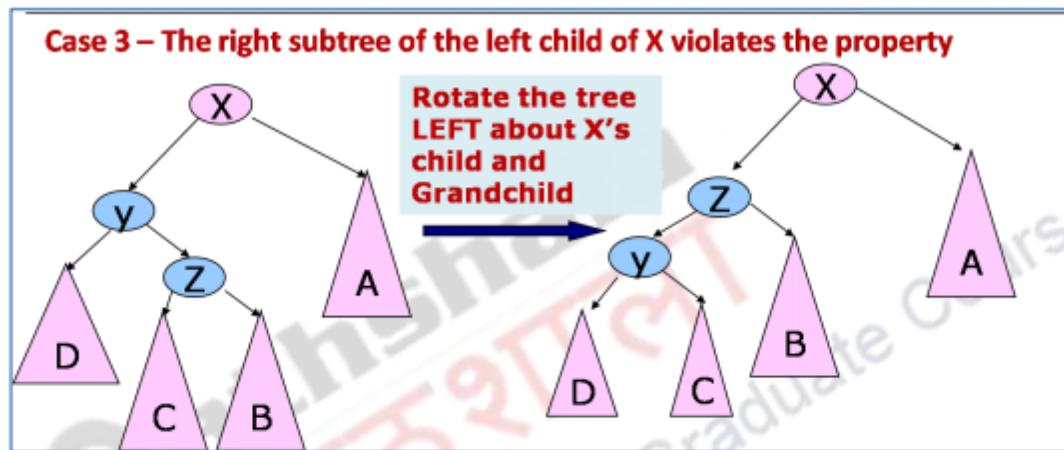
23.6.4 Double Rotations

The other two cases of rotation are cases of two single rotations. When a new item is added to the sub-tree for an inside grandchild (left-right or right-left), the imbalance is fixed with a double right or left rotation.

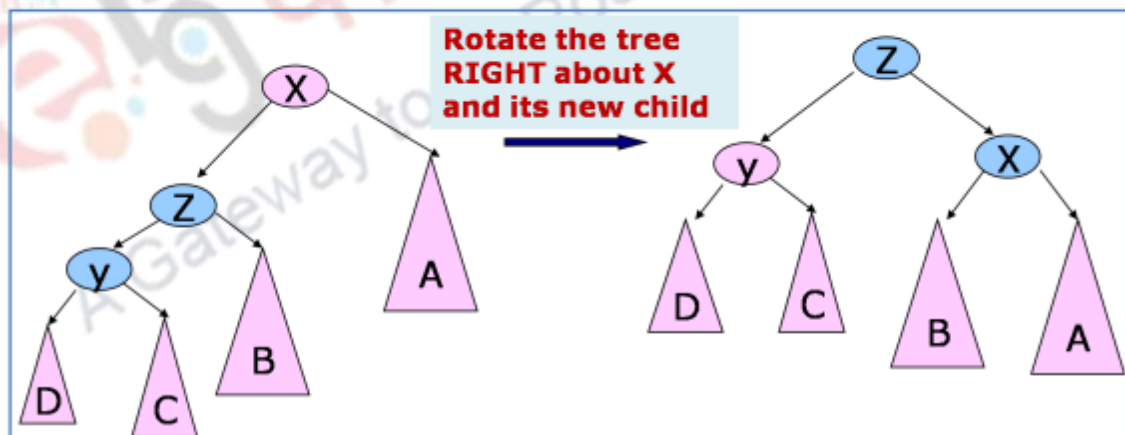
23.6.5 Left Right Double Rotation

When we have the situation where we have a left inside grandchild (left-right) case we have left right double rotation. A left-right double rotation is equivalent to

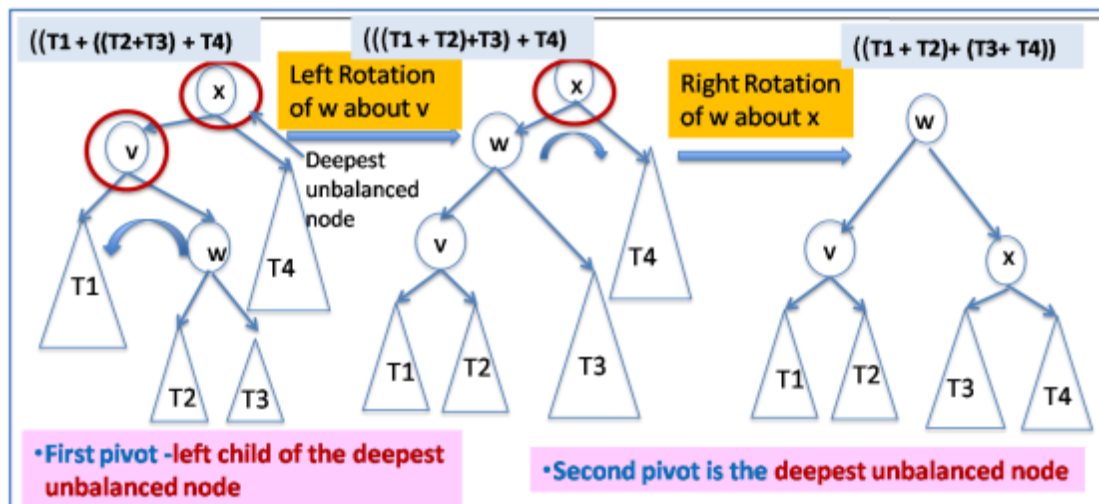
a sequence of two single rotations where the first rotation on the original tree is a *left* rotation between X's left-child and grandchild and the second rotation on this new tree is a *right* rotation between X and its new left child.



(a)



(b)



(c)

Figure 23.12 Left Right Double Rotation

Figure 23.12 shows the details of case 3. In this case the right sub-tree of the left child of X violates the property. First we need to rotate the tree LEFT about X's left child (Y) and grandchild (Z) as shown in Figure 23.12 (a) similar to single left rotation. Now we need to carry out the single right rotation of z about x in the modified tree that is we rotate the tree RIGHT about X and its new left child (Z) (Figure 23.12.(b)).

Figure 23.12 (c) shows the steps more in detail.

- The first pivot for the left rotation is the left child (v) of the deepest unbalanced node (x).
- We carry out Single Left Rotation of w about the first pivot v
- Moreover during this single left rotation the sub-trees change positions from $((T1 + ((T2+T3) + T4))$ before the first rotation to positions $((T1 + T2)+T3) + T4$ after the rotation still maintaining the BST property.
- The second pivot for the right rotation is the deepest unbalanced node itself x.
- We carry out Single Right Rotation of w about x
- Moreover during this single right rotation the sub-trees change positions from $((T1 + T2)+T3) + T4$ before the first rotation to positions $((T1 + T2)+ (T3+ T4))$ after the rotation still maintaining the BST property.

23.6.6 Right-Left Double Rotation

When we have the situation where we have a right inside grandchild (right-left) then we have right left double rotation. A right-left double rotation is equivalent to a sequence of two single rotations where the first rotation on the original tree is a right rotation between X's right-child and grandchild and the second rotation on this new tree is a left rotation between X and its new right child.

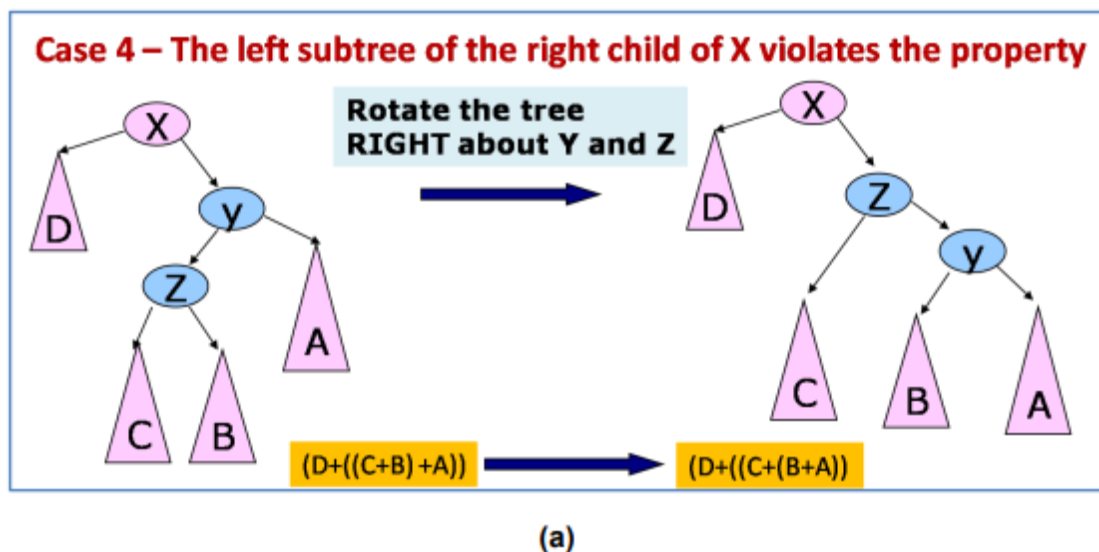
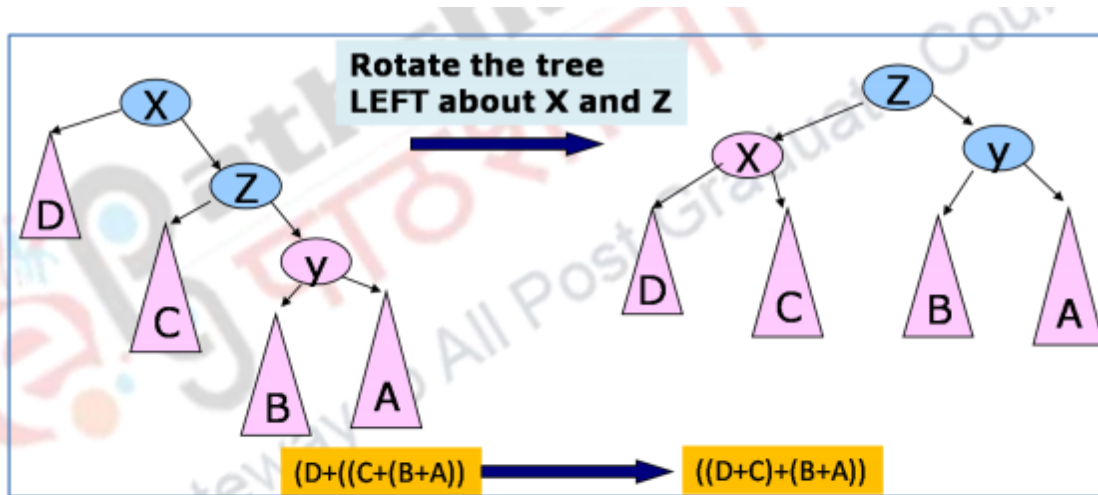
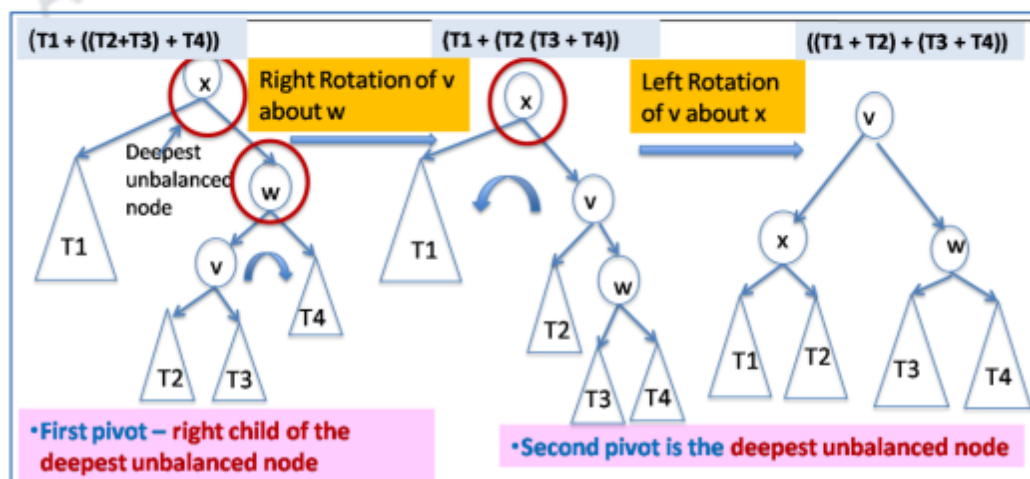


Figure 23.13 shows the details of case 4. The left sub-tree of the right child of X violates the property. First we need to rotate the tree RIGHT about X's rightchild (Y) and grandchild (Z) as shown in Figure 23.13 (a) similar to single right rotation. Now we need to carry out the single left rotation of z about x in the modified tree that is we rotate the tree LEFT about X and its new rightchild (Z) (Figure 23.13 (b)).



(b)



(c)

Figure 23.13 Right Left Double Rotation Figure 23.13 (c) shows the steps more in detail.

- The first pivot for the right rotation is right child (w) of the deepest unbalanced node (x)
- We carry out Single Right Rotation of v about the first pivot w
- Moreover during this single left rotation the sub-trees change positions from $(T1 + ((T2+T3) + T4))$ before the first rotation to positions $(T1 + (T2 (T3 + T4)))$ after the rotation still maintaining the BST property.

- The second pivot for the right rotation is the deepest unbalanced node itself x.
- We carry out Single Left Rotation of v about x
- Moreover during this single right rotation the sub-trees change positions from $(T1 + (T2 (T3 + T4)))$ before the first rotation to positions $((T1 + T2) + (T3 + T4))$ after the rotation still maintaining the BST property.

The algorithms for the double rotations are sequential call to the two appropriate single rotations with the appropriate nodes. Therefore we have seen all the possible four cases of violation and the rebalancing through rotations.

Summary

- Explained the concept of balancing and Balanced Binary Search Trees
- Discussed the properties of AVL Trees
- Described Single Rotation of AVL Trees
- Discussed Double Rotation of AVL Trees

you can view video on **Balanced Binary Search Trees and AVL Trees**



Web Links
https://courses.cs.washington.edu/courses/cse373/04wi/.../lecture08.ppt
https://www.cpp.edu/~ftang/.../trees/AVL%20tree%20animation.ppt
www.cs.fsu.edu/~asriniva/courses/DS14/lectures/Lec17-19.ppt
www.uwgb.edu/breznayp/cs351/slides/avl.ppt
Supporting & Reference Materials
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014